

REPRODUCIBILITY STUDY: ACTIVATION BY INTERVAL-WISE DROPOUT

Samuel Nicholas & James Dolan

Department of Electronics and Computer Science
University of Southampton
{sn2g22, jd7g22}@soton.ac.uk

ABSTRACT

We present a reproducibility study of Park et al. (2025), which introduces Activation by Interval-wise Dropout (AID) as a method for mitigating plasticity loss in neural networks. We re-implemented the method and its primary baselines in PyTorch, targeting the continual learning, reinforcement learning, and standard supervised learning experiments. We find that the core claim, that AID outperforms standard Dropout in maintaining plasticity, is partially reproducible, though the margin over vanilla baselines is often smaller than reported. We also propose and evaluate two extensions: SmoothAID, which replaces the hard activation threshold with a continuous sigmoid gate, and LearnableAID, which makes the dropout coefficient a learnable parameter. LearnableAID shows some improvements over AID in specific scenarios, while SmoothAID underperforms.

1 INTRODUCTION

Plasticity loss is the gradual degradation of a neural network’s ability to learn new tasks or adapt to distribution shifts during training (Dohare et al., 2024). Park et al. (2025) argue that standard Dropout fails to address this because it only creates an ensemble of subnetworks, each of which independently suffers from plasticity loss. Their proposed method, AID, applies different dropout probabilities to positive and negative preactivations, which they say acts as a regulariser pushing the network toward linear behaviour so it doesn’t suffer from plasticity loss (Lewandowski et al., 2024).

The AID operation during training is:

$$\text{AID}_p(x) = m \cdot \text{ReLU}(x) + (1 - m) \cdot (-\text{ReLU}(-x)), \quad m \sim \text{Bernoulli}(p) \quad (1)$$

and at inference it becomes a modified leaky ReLU: $r_p(x) = px$ for $x \geq 0$, $(1 - p)x$ for $x < 0$. When $p = 1$ it is standard ReLU, and when $p = 0.5$ it is a linear function, corresponding to the strongest regularisation.

2 SCOPE OF REPRODUCTION

We targeted the following experiments from the paper:

- **Section 5.1.2** - continual full and continual limited learning on CIFAR-10.
- **Section 5.2** - reinforcement learning on Arcade Learning Environment.
- **Section 5.3** - standard supervised learning on CIFAR-10, CIFAR-100 and TinyImageNet.

We did not reproduce the full set of baselines from the paper (e.g. CBP, ReDo, Fourier features) due to time and compute constraints, focusing instead on the most directly relevant comparisons: Vanilla (ReLU), Dropout, and AID, plus our two extensions. We did not train on TinyImageNet for the continual learning experiment, despite writing code for it, due to compute time constraints.

3 IMPLEMENTATION

The paper provides only the PyTorch implementation of the simplified AID module (Appendix H, five lines of code). All other code was written from scratch. We used the architecture specifications in Appendix F.3: a stripped ResNet-18 (no large-kernel stem) for CIFAR-100, a simple CNN for CIFAR-10, and VGG-16 with batch normalisation for TinyImageNet. We matched the reported hyperparameters where specified (learning rate, gradient clipping, batch size, epochs per stage) and used $p = 0.8$ for AID as stated in the paper. All experiments were run on a Nvidia L4 24GB GPU via the University of Southampton’s Iridis HPC cluster. The full codebase is available on GitHub.

4 RESULTS

4.1 CONTINUAL LEARNING (SECTION 5.1.2)

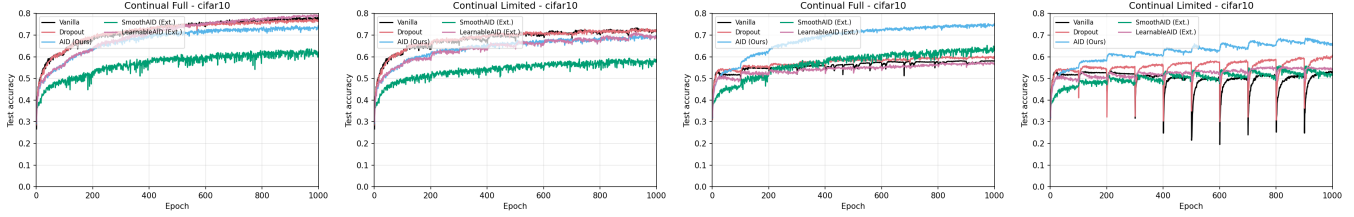


Figure 1: Test accuracy over 1000 epochs for continual full and continual limited settings on CIFAR-10 with data augmentation (left) and without (right).

Figure 1 shows the difference between continual learning results with and without data augmentation. With augmentation, vanilla, Dropout, AID, and LearnableAID all track closely in both settings, with LearnableAID performing better than AID in continual full. On the other hand, when we don't use data augmentation, we get results similar to Park et al. (2025). We believe this is partly due to data augmentation (random cropping and horizontal flipping) providing implicit regularisation that reduces the observable benefit of AID's explicit regularisation. This is consistent with findings by Ma et al. (2024), who show data augmentation can alleviate plasticity loss. Park et al. (2025) also acknowledge this, but suggest augmentation doesn't tackle the underlying cause of plasticity loss.

4.2 REINFORCEMENT LEARNING (SECTION 5.2)

We attempted to reproduce the claim from section 5.2 of the paper, that AID mitigates plasticity loss in DQN at high replay ratios (RR=1) on Arcade Learning Environment tasks. We selected the games BeamRider, DemonAttack and James Bond, as the first two were mentioned as suffering from great plasticity loss (G.4.1) and James Bond was used as a control where the paper reported neutral plasticity loss. Due to computational limitations, we chose to test the three techniques (ReLU, Dropout and AID) on these representative games averaged over 3 seeds each for 10 million frames, as running a comparison on all 17 games for 5 seeds would have taken 255 runs as opposed to 18 ($\sim 350+$ hours vs ~ 24 hours even with parallelisation). The hyperparameters for each of the techniques were taken from Table 9 in the paper. We implemented the CleanRL environment Huang et al. (2021) using the hyperparameters specified in Table 2 of Park et al. (2025) from Sokar et al. (2023).

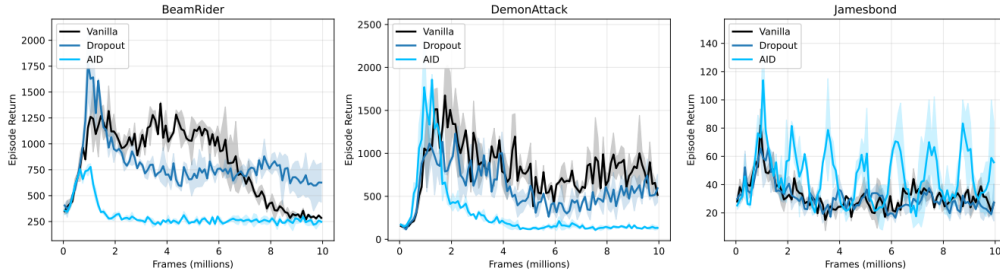


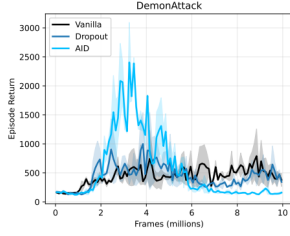
Figure 2: RL raw score on ALE games (BeamRider, DemonAttack and James Bond) vs epoch for RR=1

The results, Figure 2, we obtained by following their training techniques and the CleanRL environment setup did not match the results claimed by the paper, with each technique experiencing rapid initial growth and then catastrophic forgetting after 1-2 million frames. Vanilla and Dropout performed similarly apart from BeamRider, where Dropout outperformed vanilla, and AID consistently performed the worst except for on James Bond where it demonstrated an oscillating pattern, indicating overshooting the correct solution.

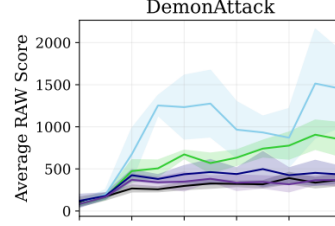
The paper claimed to be running with RR=1, however our results show a sharp training peak occurring roughly 4x sooner than the paper's results consistently across the 3 tasks. This would be consistent with their implementation running a replay ratio of RR=0.25, as they are training 4x less frequently than our implementation. AID also showed consistent collapse after 1-2 million frames, performing worse than the vanilla implementation on DemonAttack and BeamRider, which would again be consistent with the AID hyperparameter p being set too low (0.99) under the high replay ratio as the training would be aggressive. Since we were unable to reproduce their results under RR=1, this leads us to believe that the results shown in the paper were not obtained using the setup they suggested in Table 2. To investigate this, we trained with RR=0.25.

The results we obtained with RR=0.25 for DemonAttack were much more consistent with the results displayed in the paper,

with the peak of training occurring at roughly the same time, with AID achieving the highest peak score, however not mitigating plasticity loss. This could be due to our setup implementation being incorrect, however we followed their guidelines and due to the fact that their training seemed less aggressive than $RR=1$, it is unclear if their results were obtained using the parameters they claimed.



(a) RL results on DemonAttack for vanilla, Dropout and AID with $RR=0.25$



(b) RL results on DemonAttack from Park et al. (2025)

This discrepancy highlights the fragility of AID with suboptimal hyperparameter selection, which adds further motivation our extension task using a learnable hyperparameter for AID.

4.3 STANDARD SUPERVISED LEARNING (SECTION 5.3)

We reproduced the standard supervised learning experiment for section 5.3 of the paper, which evaluates if AID improves generalisation in stationary supervised settings without continual learning. We followed the original setup and compared four methods: Vanilla (ReLU), L2 weight decay, Dropout and AID across three models, each on a respective dataset. The model-dataset pairs were: 4 layer CNN on CIFAR10, ResNet-18 on CIFAR100 and VGG-16 on TinyImageNet. We trained each model for 200 epochs with the Adam optimiser with an initial learning rate of $1e-3$, decayed by a factor of 10 at epoch 100 and epoch 150, according to Appendix F.5 of the paper. The hyperparameters for AID, Dropout and L2 were taken from Table 10 of the paper. We also replaced the max pooling with average pooling in the CNN and VGG-16 architectures with AID since AID would pass negative pre activations that max pooling would discard.

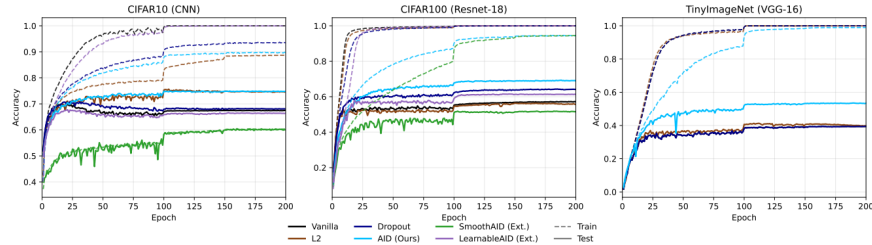


Figure 4: Training and test accuracy over 200 epochs on CIFAR-10 (CNN), CIFAR-100 (ResNet-18) and TinyImageNet (VGG-16). Solid lines are test accuracy, dashed lines are train accuracy. LR is divided by 10 at epochs 100 and 150.

Table 1 shows the final test accuracies. We were able to successfully replicate the original paper’s Table 1 demonstrating that AID matches or outperforms other regularisation techniques on all three datasets. The largest gains were with ResNet-18 on CIFAR-100 and VGG-16 on TinyImageNet.

Table 1: Final test accuracy for each method on the standard supervised learning benchmark (Section 5.3).

Method	CIFAR-10 (CNN)	CIFAR-100 (ResNet-18)	TinyImageNet (VGG-16)
Vanilla	0.675	0.572	0.395
L2	0.746	0.557	0.398
Dropout	0.681	0.641	0.395
AID	0.748	0.689	0.534
SmoothAID (Ext.)	0.601	0.515	—
LearnableAID (Ext.)	0.664	0.614	—

The supervised learning results (Figure 4) show the learning curves, with AID maintaining a smaller train/test gap than the other techniques, which supports the paper’s claim that AID performs well in stationary supervised settings. Additionally, the LR decay at epoch 100 causes a sharp jump in accuracy for most methods, which is expected as per the results in the paper. In this case LearnableAID underperforms compared to vanilla AID, which could potentially be due to the straight-through gradient estimator it uses to make p

learnable. The forward pass samples a hard Bernoulli mask while gradients are routed through the smooth expectation, making the weight updates biased over the 200 training epochs.

5 EXTENSIONS

We proposed two variants of AID as part of this paper:

SmoothAID replaces the hard threshold at $x = 0$ with a sigmoid gate: $p_{\text{eff}}(x) = p \cdot \sigma(\text{sharpness} \cdot x)$. The motivation was that the hard boundary in AID could produce suboptimal gradients near zero. In practice, SmoothAID consistently underperformed across all experiments, with notably higher variance in the continual setting. This is an informative negative result: The smooth transition appears to weaken the regularisation effect that makes AID work, rather than improving it.

LearnableAID stores p as an unconstrained scalar p_{raw} and projects it via sigmoid during the forward pass, so the network learns how strongly to regularise toward linear behaviour. Since the stochastic mask is non-differentiable with respect to p , we used a straight-through estimator. The stochastic output is returned, but a zero-valued differentiable term ($\hat{y} - \hat{y}.\text{detach}()$) is added, where $\hat{y}$ is the deterministic leaky-ReLU output, providing a gradient path to p_{raw} . LearnableAID outperformed AID in Continual Full testing on cifar-10 with data augmentation, but underperformed without. This suggests that allowing the network to self-regulate its degree of linear regularisation is beneficial under certain test conditions, and removes the need to tune p manually.

6 DISCUSSION AND CONCLUSIONS

The central claim of Park et al. (2025), that AID outperforms Dropout in maintaining plasticity, is partially reproducible. We find consistent evidence of AID outperforming Dropout in the continual CIFAR-10 setting (without data augmentation), which aligns with the paper. It is also worth noting that applying data augmentation makes AID perform the same as the vanilla models, reducing the need for AID in many circumstances. Additionally, we were unable to fully reproduce the paper’s results on the RL tasks, and we found their results to be more consistent with a different replay ratio than stated in the paper. These factors indicate that the AID technique is highly sensitive to hyperparameter settings which need to be tailored for the specific task. We suspect the paper tuned hyperparameters extensively over multiple seeds (Tables 3-10 of the appendix), whereas (in continual learning) we used a single seed and a fixed p .

Reproducing this paper from scratch required roughly a month of development effort. The main constraint was on compute time, which spanned from 8 to 12 hours on a single GPU on Iridis for cifar10, and 12-20 hours for cifar100. The paper’s full experimental suite (all datasets, all baselines, five seeds) would require substantially more compute than was available to us.

The paper’s core theoretical contribution (Theorem 4.1, showing AID has a regularisation effect toward linear networks) is not something we verified empirically beyond a basic numerical sanity check in our unit tests. The theoretical analysis is likely the more robust contribution. Our results suggest the effect exists but may be more sensitive to implementation details than the paper implies.

REFERENCES

- Shibhansh Dohare, J. Fernando Hernandez-Garcia, Parash Rahman, A. Rupam Mahmood, and Richard S. Sutton. Maintaining plasticity in deep continual learning, 2024. URL <https://arxiv.org/abs/2306.13812>.
- Shengyi Huang, Rousslan Fernand Julien Dossa, Chang Ye, and Jeff Braga. Cleanrl: High-quality single-file implementations of deep reinforcement learning algorithms, 2021. URL <https://arxiv.org/abs/2111.08819>.
- Alex Lewandowski, Dale Schuurmans, and Marlos C. Machado. Plastic learning with deep fourier features, 2024. URL <https://arxiv.org/abs/2410.20634>.
- Guozheng Ma, Lu Li, Sen Zhang, Zixuan Liu, Zhen Wang, Yixin Chen, Li Shen, Xueqian Wang, and Dacheng Tao. Revisiting plasticity in visual reinforcement learning: Data, modules and training stages, 2024. URL <https://arxiv.org/abs/2310.07418>.
- Sangyeon Park, Isaac Han, Seungwon Oh, and Kyung-Joong Kim. Activation by interval-wise dropout: A simple way to prevent neural networks from plasticity loss, 2025. URL <https://arxiv.org/abs/2502.01342>.
- Ghada Sokar, Rishabh Agarwal, Pablo Samuel Castro, and Utku Evci. The dormant neuron phenomenon in deep reinforcement learning, 2023. URL <https://arxiv.org/abs/2302.12902>.